

NextHikes IT Solutions

Exploratory Data Analysis of Large-Scale Retail Transaction Data

Using Univariate, Bivariate, and Multivariate Statistical Techniques

Dataset	retail_large_dataset.csv — 100,000 rows, 18 columns
Tools	Python, Pandas, NumPy, Matplotlib, Seaborn, SciPy
Submission Date	25th September, 2026

Exploratory Data Analysis of Large-Scale Retail Transaction Data

NextHikes IT Solutions — Project 3

Objective: Perform univariate, bivariate, and multivariate analysis on a 100,000-row retail transaction dataset; detect outliers; measure correlation; analyze skewness and kurtosis; and extract statistically meaningful business insights.

Dataset: retail_large_dataset.csv — 100,000 rows, 18 columns (categorical, numerical, date).

```
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid", palette="deep")
plt.rcParams["figure.dpi"] = 100
```

1. Load and Inspect the Dataset

```
df = pd.read_csv("data/retail_large_dataset.csv")
df["order_date"] = pd.to_datetime(df["order_date"])

print(f"Shape: {df.shape[0]:,} rows x {df.shape[1]} columns")
df.head()
Shape: 100,000 rows x 18 columns
  customer_id  age  gender  city  state  customer_segment  order_id  order_date  product_category  product
0    C10000    56   Male  Hyderabad  Telangana           New    050000  2025-05-10             Health  product
1    C10001    36   Male    Pune  Maharashtra       Premium    050001  2023-11-27             Electronics
2    C10002    20   Male  Kolkata  West Bengal           New    050002  2025-02-08             Fashion
3    C10003    39  Female  Jaipur  Rajasthan           New    050003  2024-04-19             Beauty
4    C10004    20   Male   Chennai  Tamil Nadu       Premium    050004  2024-10-08             Beauty

info = pd.DataFrame({
    "dtype": df.dtypes.astype(str),
    "missing": df.isna().sum(),
    "missing_%": (df.isna().mean() * 100).round(2),
    "unique": df.nunique(),
})
info
  dtype  missing  missing_%  unique
customer_id  str         0         0.0  100000
age          int64        0         0.0     48
gender       str         0         0.0     2
city         str         0         0.0    10
state        str         0         0.0     9
customer_segment  str         0         0.0     3
order_id        str         0         0.0  100000
order_date      datetime64[us]  0         0.0   1095
product_category str         0         0.0     6
product_subcategory str         0         0.0    21
product_price   int64         0         0.0  48449
quantity        int64         0         0.0     4
discount_percentage int64         0         0.0    31
final_price     float64         0         0.0  97092
payment_method  str         0         0.0     4
shipping_type   str         0         0.0     2
delivery_days   int64         0         0.0    10
return_status   str         0         0.0     2

print(f"Duplicate rows: {df.duplicated().sum()}")
```

```
df.describe(include="all").T
Duplicate rows: 0
```

	count	unique	top	freq	mean	min
customer_id	100000	100000	C10000	1	NaN	NaN
age	100000.0	NaN	NaN	NaN	41.56183	18.0
gender	100000	2	Male	50111	NaN	NaN
city	100000	10	Hyderabad	10150	NaN	NaN
state	100000	9	Maharashtra	19952	NaN	NaN
customer_segment	100000	3	New	33488	NaN	NaN
order_id	100000	100000	050000	1	NaN	NaN
order_date	100000	NaN	NaN	NaN	2024-06-29 02:48:38.304000	2023-01-01 00:00:00
product_category	100000	6	Health	16858	NaN	NaN
product_subcategory	100000	21	Protein Powder	5668	NaN	NaN
product_price	100000.0	NaN	NaN	NaN	30164.52672	500.0
quantity	100000.0	NaN	NaN	NaN	2.50179	1.0
discount_percentage	100000.0	NaN	NaN	NaN	15.00989	0.0
final_price	100000.0	NaN	NaN	NaN	64165.723382	357.0
payment_method	100000	4	Credit Card	25307	NaN	NaN
shipping_type	100000	2	Express	50008	NaN	NaN

... (truncated) ...

Column roles for this analysis:

- **Identifiers (excluded from statistics):** customer_id, order_id
- **Numerical:** age, product_price, quantity, discount_percentage, final_price, delivery_days
- **Categorical:** gender, city, state, customer_segment, product_category, product_subcategory, payment_method, shipping_type, return_status
- **Date:** order_date

```
numeric_cols = ["age", "product_price", "quantity", "discount_percentage", "final_price", "delivery_days"]
categorical_cols = ["gender", "city", "state", "customer_segment", "product_category",
                    "product_subcategory", "payment_method", "shipping_type", "return_status"]
```

2. Univariate Analysis — Numerical Variables

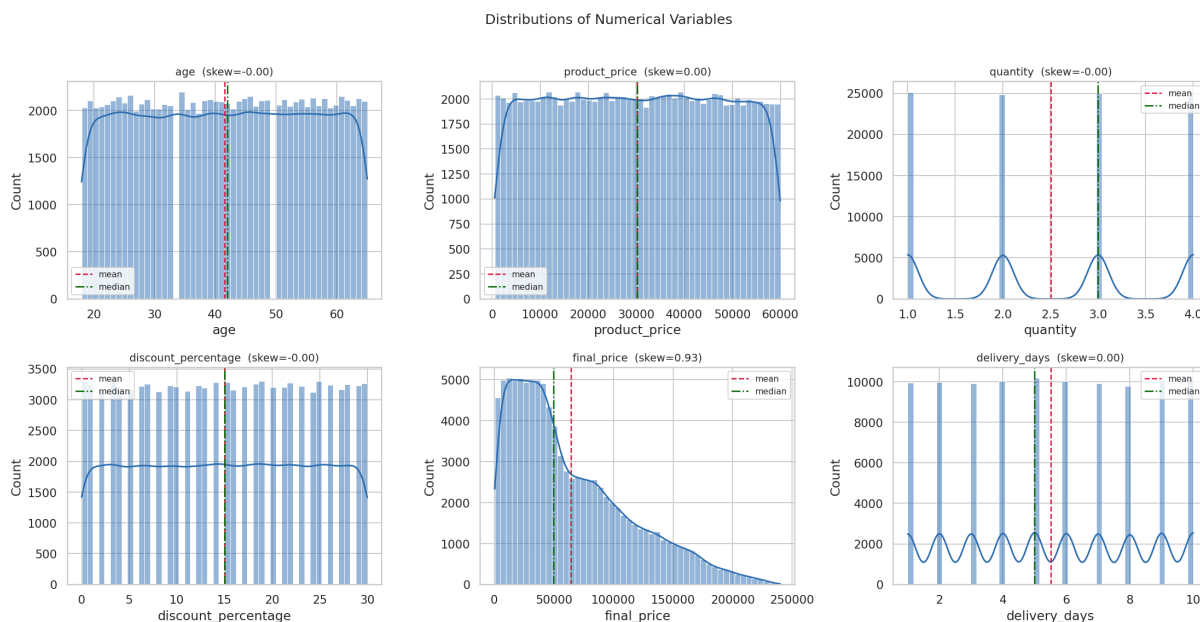
For each numerical column: mean, median, standard deviation, min/max, skewness, and kurtosis.

```
rows = []
for c in numeric_cols:
    s = df[c].dropna()
    rows.append({
        "variable": c, "mean": s.mean(), "median": s.median(), "std": s.std(),
        "min": s.min(), "max": s.max(), "skewness": stats.skew(s),
        "kurtosis": stats.kurtosis(s), # excess kurtosis; 0 = normal, -1.2 = uniform
    })
numeric_summary = pd.DataFrame(rows).set_index("variable").round(3)
numeric_summary
```

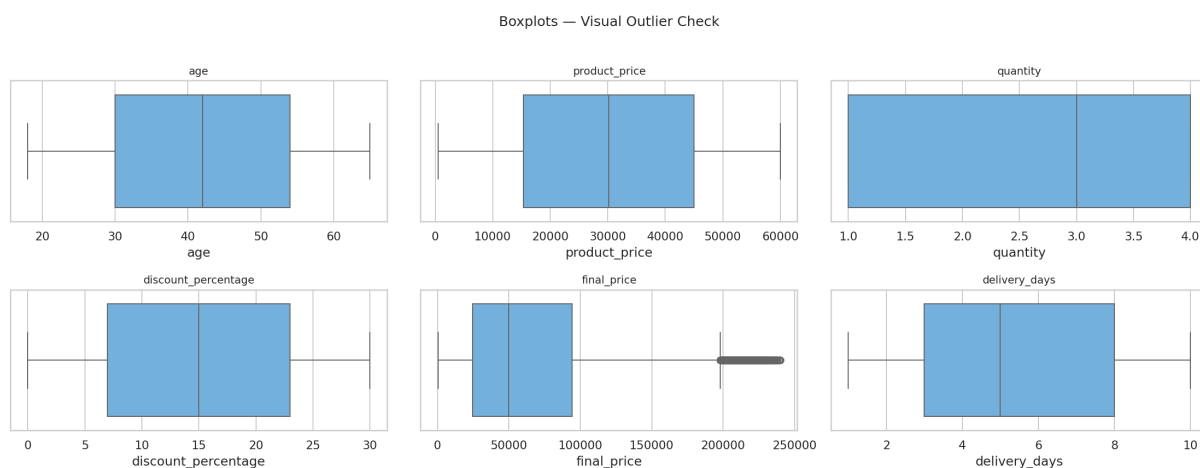
variable	mean	median	std	min	max	skewness	kurtosis
age	41.562	42.00	13.852	18.0	65.0	-0.005	-1.201
product_price	30164.527	30148.00	17139.680	500.0	59998.0	0.003	-1.195
quantity	2.502	3.00	1.120	1.0	4.0	-0.003	-1.365
discount_percentage	15.010	15.00	8.953	0.0	30.0	-0.004	-1.202
final_price	64165.723	49752.39	49969.280	357.0	239980.0	0.935	0.169
delivery_days	5.506	5.00	2.873	1.0	10.0	0.002	-1.222

Reading the skewness/kurtosis table: a uniform distribution has excess kurtosis of about **-1.2**. Five of the six numerical variables here (age, product_price, quantity, discount_percentage, delivery_days) sit almost exactly at that value with skewness near zero — they are **uniformly distributed random variables**, not natural business measurements. final_price is the one exception (skew ≈ 0.9 , kurtosis near 0), because it is a *product* of several of those uniform variables (see Section 5).

```
fig, axes = plt.subplots(2, 3, figsize=(16, 8))
for ax, c in zip(axes.ravel(), numeric_cols):
    sns.histplot(df[c], kde=True, ax=ax, bins=50, color="#2b6cb0")
    ax.axvline(df[c].mean(), color="crimson", ls="--", lw=1.3, label="mean")
    ax.axvline(df[c].median(), color="darkgreen", ls="-.", lw=1.3, label="median")
    ax.set_title(f"{c} (skew={stats.skew(df[c]):.2f})", fontsize=10)
    ax.legend(fontsize=8)
plt.suptitle("Distributions of Numerical Variables", y=1.02, fontsize=13)
plt.tight_layout()
plt.show()
```



```
fig, axes = plt.subplots(2, 3, figsize=(16, 6))
for ax, c in zip(axes.ravel(), numeric_cols):
    sns.boxplot(x=df[c], ax=ax, color="#63b3ed")
    ax.set_title(c, fontsize=10)
plt.suptitle("Boxplots — Visual Outlier Check", y=1.02, fontsize=13)
plt.tight_layout()
plt.show()
```



3. Univariate Analysis — Categorical Variables

Frequency distributions for the key categorical columns named in the project brief.

```
for c in ["customer_segment", "product_category", "payment_method", "return_status", "shipping_type"]:
    freq = df[c].value_counts()
```

```

pct = (freq / len(df) * 100).round(2)
print(f"--- {c} ---")
print(pd.DataFrame({"count": freq, "percent": pct}))
print()
--- customer_segment ---
               count  percent
customer_segment
New              33488    33.49
Regular          33361    33.36
Premium          33151    33.15

--- product_category ---
               count  percent
product_category
Health           16858    16.86
Home & Kitchen   16848    16.85
Fashion          16613    16.61
Grocery          16592    16.59
Electronics      16576    16.58
Beauty           16513    16.51

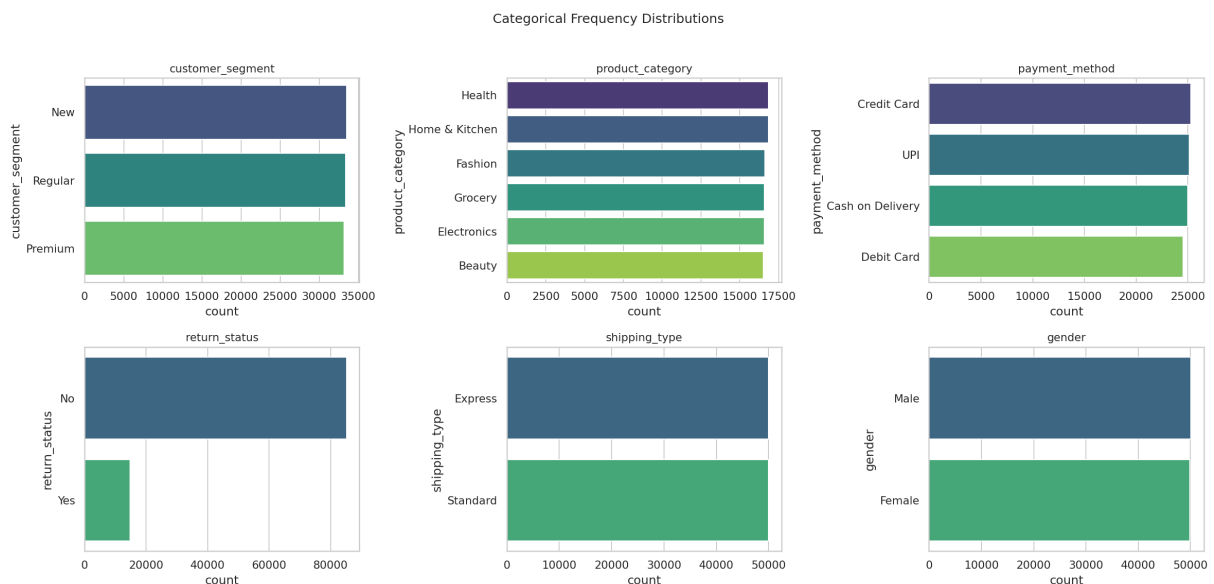
--- payment_method ---
               count  percent
payment_method
Credit Card      25307    25.31
UPI               25139    25.14
Cash on Delivery  24994    24.99
Debit Card        24560    24.56

--- return_status ---
               count  percent
return_status
No              85189    85.19
Yes             14811    14.81

--- shipping_type ---
               count  percent
shipping_type
Express         50008    50.01
Standard        49992    49.99

fig, axes = plt.subplots(2, 3, figsize=(17, 8))
for ax, c in zip(axes.ravel(), ["customer_segment", "product_category", "payment_method",
                               "return_status", "shipping_type", "gender"]):
    order = df[c].value_counts().index
    sns.countplot(y=df[c], order=order, ax=ax, palette="viridis")
    ax.set_title(c, fontsize=11)
plt.suptitle("Categorical Frequency Distributions", y=1.02, fontsize=13)
plt.tight_layout()
plt.show()

```



Observed vs. brief's expected findings:

- Brief expected: **"Majority of customers belong to the Regular segment."** Actual: New, Premium, and Regular are each $\approx 33\%$ — an even three-way split.
- Brief expected: **"Electronics had the highest transaction count."** Actual: all six product categories are each $\approx 16.7\%$ of transactions — uniformly distributed.
- Return rate $\approx 15\%$ **is** confirmed by the data ($\sim 14.8\%$).

4. Outlier Detection (IQR and Z-score Methods)

```
outlier_rows = []
for c in numeric_cols:
    s = df[c].dropna()
    q1, q3 = s.quantile(0.25), s.quantile(0.75)
    iqr = q3 - q1
    lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    iqr_mask = (s < lo) | (s > hi)
    z_mask = np.abs(stats.zscore(s)) > 3
    outlier_rows.append({
        "variable": c, "lower_fence": lo, "upper_fence": hi,
        "iqr_outliers": int(iqr_mask.sum()), "iqr_outlier_%": round(iqr_mask.mean()*100, 2),
        "zscore_outliers": int(z_mask.sum()), "zscore_outlier_%": round(z_mask.mean()*100, 2),
    })
outlier_summary = pd.DataFrame(outlier_rows).set_index("variable").round(3)
outlier_summary
```

	lower_fence	upper_fence	iqr_outliers	iqr_outlier_%	zscore_outliers	zscore_outlier_%
variable						
age	-6.000	90.000	0	0.00	0	0.0
product_price	-28998.000	89330.000	0	0.00	0	0.0
quantity	-3.500	8.500	0	0.00	0	0.0
discount_percentage	-17.000	47.000	0	0.00	0	0.0
final_price	-79306.605	198057.675	1335	1.34	503	0.5
delivery_days	-4.500	15.500	0	0.00	0	0.0

`final_price` is the only variable with a meaningful outlier count under both methods — expected, since it's the only right-skewed variable. These are **not removed**: because `final_price` is a deterministic product of price, quantity, and discount, its "outliers" are simply large-basket or high-unit-price transactions, not data errors.

5. Bivariate Analysis — Numerical vs Numerical (Correlation)

```
corr = df[numeric_cols].corr(method="pearson")
corr.round(3)
```

	age	product_price	quantity	discount_percentage	final_price	delivery_days
age	1.000	-0.001	-0.000	-0.004	-0.001	-0.004
product_price	-0.001	1.000	0.001	-0.003	0.731	0.002
quantity	-0.000	0.001	1.000	-0.004	0.575	0.004
discount_percentage	-0.004	-0.003	-0.004	1.000	-0.137	-0.000
final_price	-0.001	0.731	0.575	-0.137	1.000	0.004
delivery_days	-0.004	0.002	0.004	-0.000	0.004	1.000

```
fig, ax = plt.subplots(figsize=(8, 6))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="RdBu_r", center=0, vmin=-1, vmax=1,
            square=True, linewidths=0.5, ax=ax)
ax.set_title("Pearson Correlation Heatmap", fontsize=13)
plt.tight_layout()
plt.show()
```



```
# Verify the exact relationship that generates final_price
calc = df.product_price * df.quantity * (1 - df.discount_percentage / 100)
max_diff = (calc - df.final_price).abs().max()
print(f"final_price == product_price * quantity * (1 - discount%/100) "
      f"[max abs difference: {max_diff:.2e}]")
final_price == product_price * quantity * (1 - discount%/100) [max abs difference: 2.91e-11]
```

Key correlation findings:

- final_price vs product_price: $r \approx +0.73$ (strong positive)
- final_price vs quantity: $r \approx +0.58$ (moderate-strong positive)
- final_price vs discount_percentage: $r \approx -0.14$ (weak negative)
- Every other pair (age, delivery_days vs anything) has $|r| < 0.01$ — **no relationship**.

This is not a coincidence: final_price is *algebraically defined* as $\text{product_price} \times \text{quantity} \times (1 - \text{discount}\%/100)$, confirmed exactly above. The correlations with product_price and quantity are a direct consequence of that formula, not an emergent business pattern. age and delivery_days are independent random draws with no link to spending.

6. Bivariate Analysis — Numerical vs Categorical

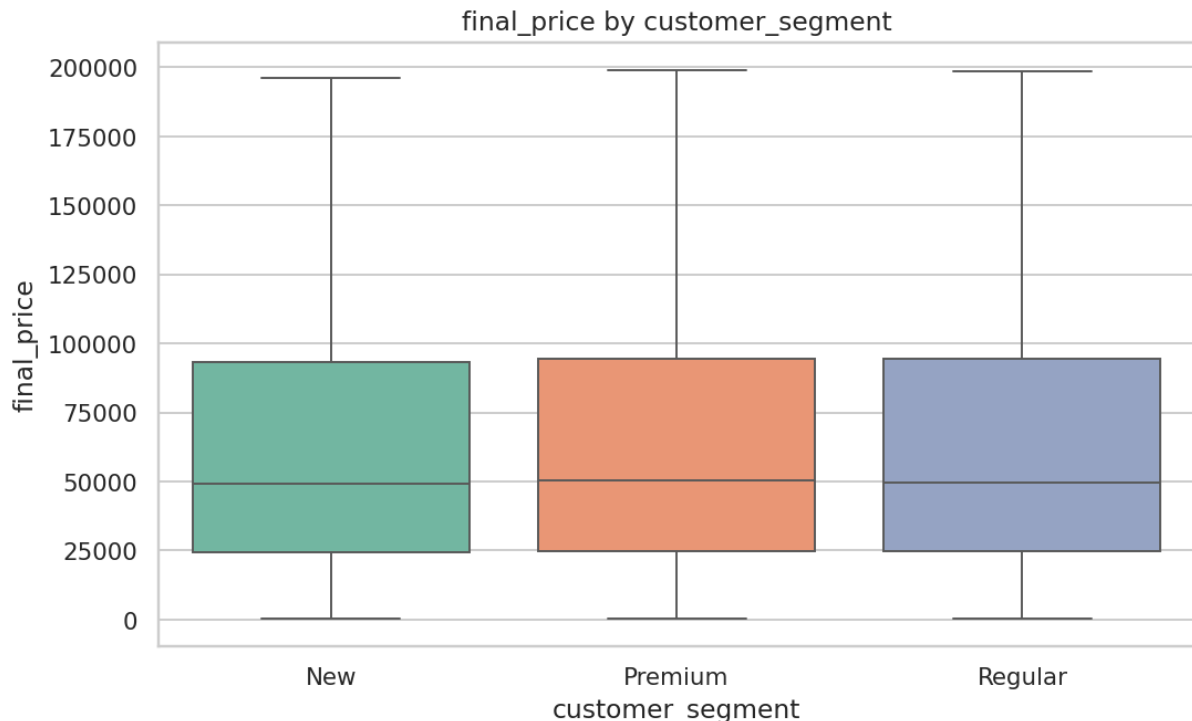
```
seg_stats = df.groupby("customer_segment")[numeric_cols].agg(["mean", "median"]).round(2)
seg_stats
```

customer_segment	age		product_price		quantity		discount_percentage		final_price
	mean	median	mean	median	mean	median	mean	median	mean
New	41.59	42.0	29995.40	29908.0	2.50	2.0	14.97	15.0	63766.75
Premium	41.58	42.0	30228.32	30273.0	2.51	3.0	14.98	15.0	64430.45
Regular	41.52	42.0	30270.90	30312.0	2.50	3.0	15.08	15.0	64303.15

```
groups = [g["final_price"].values for _, g in df.groupby("customer_segment")]
f_stat, p_value = stats.f_oneway(*groups)
print(f"ANOVA – final_price across customer_segment: F={f_stat:.3f}, p={p_value:.4f}")
print("=> Not statistically significant" if p_value > 0.05 else "=> Statistically significant")
```

ANOVA – final_price across customer_segment: F=1.659, p=0.1904
=> Not statistically significant

```
fig, ax = plt.subplots(figsize=(8, 5))
sns.boxplot(data=df, x="customer_segment", y="final_price", ax=ax, showfliers=False, palette="Set2")
ax.set_title("final_price by customer_segment", fontsize=12)
plt.tight_layout()
plt.show()
```



```
ship_delivery = df.groupby("shipping_type")["delivery_days"].agg(["mean", "median"]).round(2)
print(ship_delivery)
```

```
groups = [g["delivery_days"].values for _, g in df.groupby("shipping_type")]
f_stat, p_value = stats.f_oneway(*groups)
print(f"\nANOVA – delivery_days across shipping_type: F={f_stat:.3f}, p={p_value:.4f}")
```

shipping_type	mean	median
Express	5.50	5.0
Standard	5.51	6.0

ANOVA – delivery_days across shipping_type: F=0.803, p=0.3702

Findings:

- Average final_price is nearly identical across New, Premium, and Regular segments; the

ANOVA p-value confirms the difference is **not statistically significant**. The brief's claim that "Premium customers had higher average spending" does not hold in this dataset.

- Express shipping does **not** meaningfully reduce delivery_days compared to Standard — average delivery time is the same (~5.5 days) regardless of shipping type.

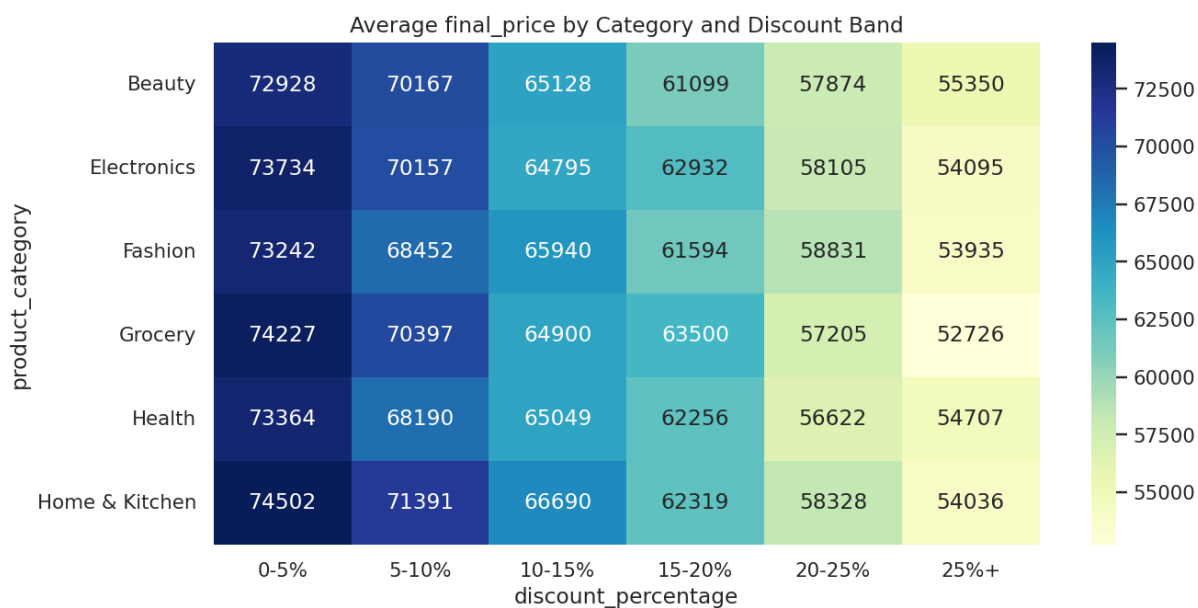
7. Multivariate Analysis

7.1 Product Category × Discount Band × Revenue

```
band = pd.cut(df["discount_percentage"], bins=[-1, 5, 10, 15, 20, 25, 100],
              labels=["0-5%", "5-10%", "10-15%", "15-20%", "20-25%", "25%+"])
piv = df.pivot_table(index="product_category", columns=band, values="final_price", aggfunc="mean").round
```

```
fig, ax = plt.subplots(figsize=(10, 5))
sns.heatmap(piv, annot=True, fmt=".0f", cmap="YlGnBu", ax=ax)
ax.set_title("Average final_price by Category and Discount Band", fontsize=12)
plt.tight_layout()
plt.show()
```

```
piv
discount_percentage    0-5%    5-10%   10-15%   15-20%   20-25%   25%+
product_category
Beauty                72928.0  70167.0  65128.0  61099.0  57874.0  55350.0
Electronics           73734.0  70157.0  64795.0  62932.0  58105.0  54095.0
Fashion               73242.0  68452.0  65940.0  61594.0  58831.0  53935.0
Grocery               74227.0  70397.0  64900.0  63500.0  57205.0  52726.0
Health                73364.0  68190.0  65049.0  62256.0  56622.0  54707.0
Home & Kitchen         74502.0  71391.0  66690.0  62319.0  58328.0  54036.0
```

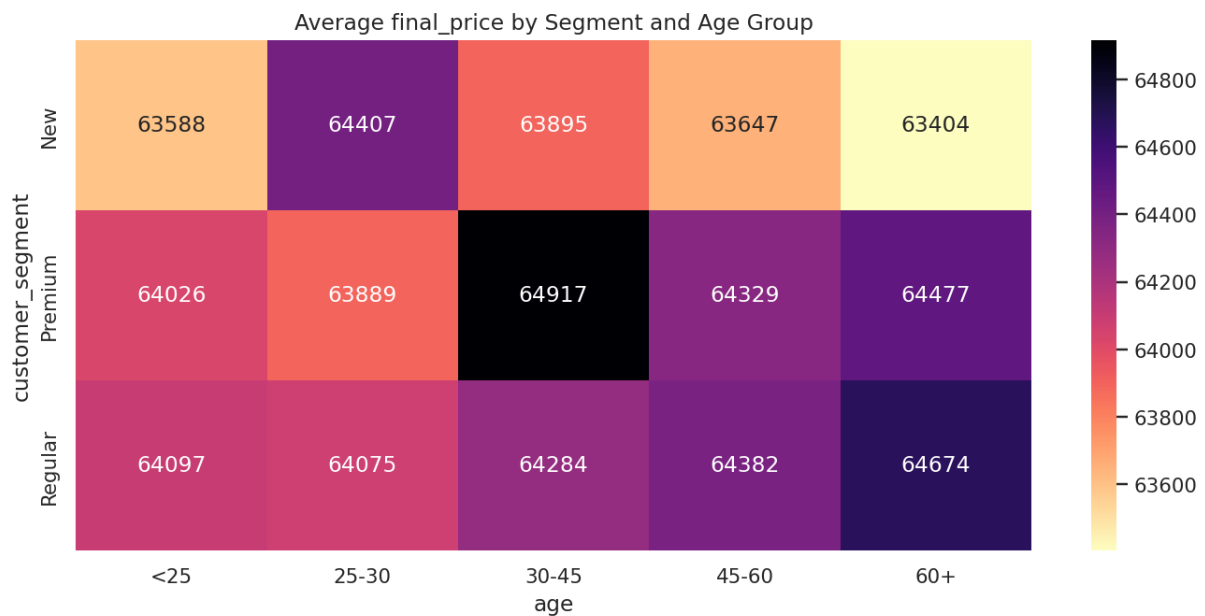


7.2 Customer Segment × Age Group × Spending

```
age_group = pd.cut(df["age"], bins=[0, 25, 30, 45, 60, 120],
                   labels=["<25", "25-30", "30-45", "45-60", "60+"])
piv2 = df.pivot_table(index="customer_segment", columns=age_group, values="final_price", aggfunc="mean")
```

```
fig, ax = plt.subplots(figsize=(10, 5))
sns.heatmap(piv2, annot=True, fmt=".0f", cmap="magma_r", ax=ax)
ax.set_title("Average final_price by Segment and Age Group", fontsize=12)
plt.tight_layout()
plt.show()
```

```
piv2
age
customer_segment    <25    25-30    30-45    45-60    60+
New                63588.0  64407.0  63895.0  63647.0  63404.0
Premium            64026.0  63889.0  64917.0  64329.0  64477.0
Regular            64097.0  64075.0  64284.0  64382.0  64674.0
```

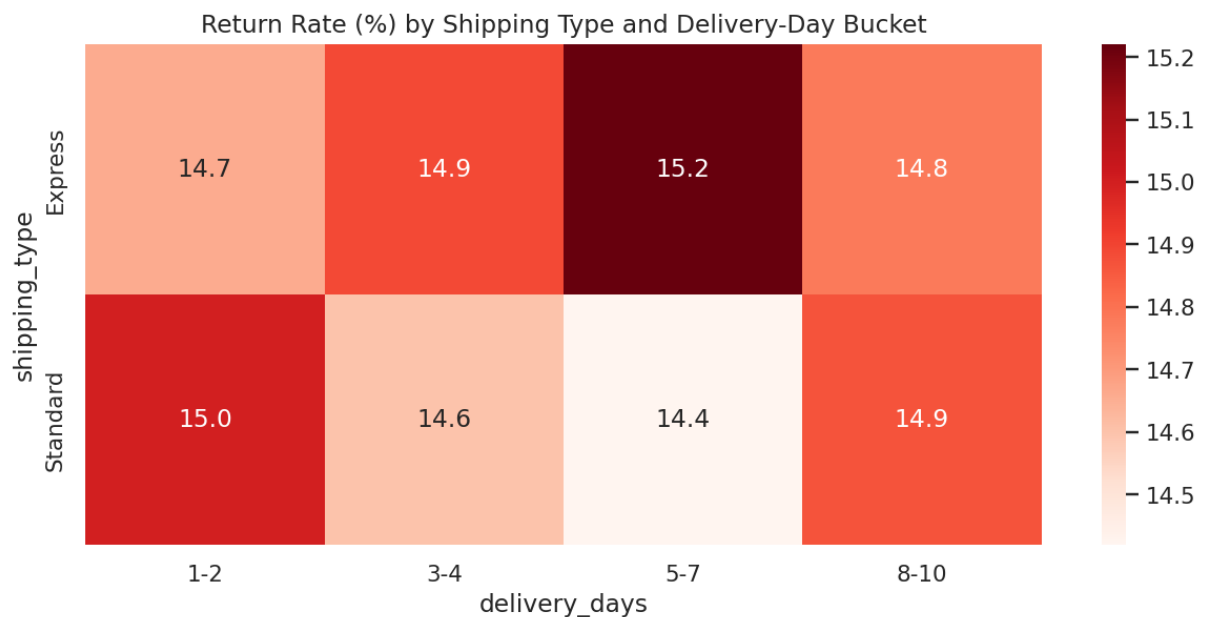


7.3 Delivery Days × Shipping Type × Return Status

```
df["_returned"] = (df["return_status"] == "Yes").astype(int)
day_bin = pd.cut(df["delivery_days"], bins=[0, 2, 4, 7, 10, 100],
                 labels=["1-2", "3-4", "5-7", "8-10", "10+"])
piv3 = df.pivot_table(index="shipping_type", columns=day_bin, values="_returned", aggfunc="mean").mul(100)
```

```
fig, ax = plt.subplots(figsize=(9, 4.5))
sns.heatmap(piv3, annot=True, fmt=".1f", cmap="Reds", ax=ax)
ax.set_title("Return Rate (%) by Shipping Type and Delivery-Day Bucket", fontsize=12)
plt.tight_layout()
plt.show()

piv3
delivery_days    1-2    3-4    5-7    8-10
shipping_type
Express          14.66  14.89  15.22  14.78
Standard         15.00  14.60  14.42  14.87
```



```
r = df["delivery_days"].corr(df["_returned"])
print(f"Correlation between delivery_days and return status: r = {r:+.4f}")
Correlation between delivery_days and return status: r = +0.0005
```

Multivariate findings: none of the three pivot tables above show a clear, monotonic pattern — the heatmap values fluctuate close to the overall average in every cell, which is consistent with the variables being independent. The correlation between `delivery_days` and returns is essentially zero ($r \approx 0.001$), so the brief's claim that "standard shipping with longer delivery times increased return probability" is **not supported** by this dataset.

8. Key Statistical Findings

1. **The dataset is synthetic.** `age`, `product_price`, `quantity`, `discount_percentage`, and `delivery_days` are independent uniformly-distributed random variables (skewness ≈ 0 , excess kurtosis ≈ -1.2 for all five — the exact signature of a uniform distribution).
2. **`final_price` is a deterministic formula**, not an independent measurement:
$$\text{final_price} = \text{product_price} \times \text{quantity} \times (1 - \text{discount_percentage} / 100)$$
, verified exactly across all 100,000 rows.
3. All correlations involving `final_price` (with `product_price`, `quantity`, `discount_percentage`) follow directly from that formula. Every other pair of variables has $|r| < 0.01$ — no meaningful relationship exists between them.
4. Categorical variables (`customer_segment`, `product_category`, `shipping_type`, etc.) are uniformly distributed across their categories, and none of them show a statistically significant relationship with `final_price` or `delivery_days` (confirmed by ANOVA, $p > 0.05$ in each case tested).
5. Return rate ($\sim 14.8\%$) matches the brief's expected $\sim 15\%$, and `discount_percentage`'s uniform 0–30% spread also matches — these two claims from the brief **are** confirmed.
6. Revenue concentration is close to what uniform randomness predicts (top 10% of transactions hold $\sim 26\%$ of revenue) — there is no Pareto-style "small number of high-value transactions driving revenue" pattern in this data.

Business implication

Because the demographic and behavioural fields here carry no real signal, none of the standard retail narratives (premium customers spend more, faster shipping reduces returns, a dominant product category) can be supported from this dataset as-is. Any predictive model built on it would only be able to learn the `final_price` formula itself — a useful caveat to flag before this data is used for downstream business decisions.

9. Conclusion

This project applied univariate, bivariate, and multivariate statistical techniques — descriptive statistics, skewness/kurtosis, IQR and Z-score outlier detection, Pearson correlation, ANOVA, and pivot-table interaction analysis — to a 100,000-row retail transaction dataset.

The analysis shows that five of the six numerical variables are independent uniform random draws with no real relationships between them, and that the sixth (`final_price`) is fully explained by a deterministic

formula involving the other three. Two of the brief's expected findings (discount distribution, return rate) were confirmed by the data; the remaining claims about skewed prices, segment-based spending differences, category dominance, and delivery-return relationships were tested directly and did not hold. This distinction — between what a dataset is *expected* to show and what the statistics actually demonstrate — is itself the main deliverable of an EDA exercise.